Curso online: **Programación orientada a objetos: Java avanzado**

Ejercicio 2

Implementación de una aplicación web que tome datos y procese dichos datos.

Autores

Aurelio García Prieto

Miguel Sallent Sánchez

ÍNDICE

Introducción.....3

RECORDATORIO: flujo de datos.....5

PASO 0. Preparación inicial.....6

PASO 1. Ampliación del modelo de dominio.....7

PASO 2. Adaptación del DAO.....7

PASO 3. Creación y actualización de clientes.....8

PASO 4. Modificación de la vista inicial (listado de clientes).....9

PASO 5. Implementación del listado de pedidos.....9

PASO 6. Creación de pedidos.....11

PASO 7. Pruebas.....12

PASO 8. Opcional. Se incluirán las validaciones en los campos.....12

Introducción

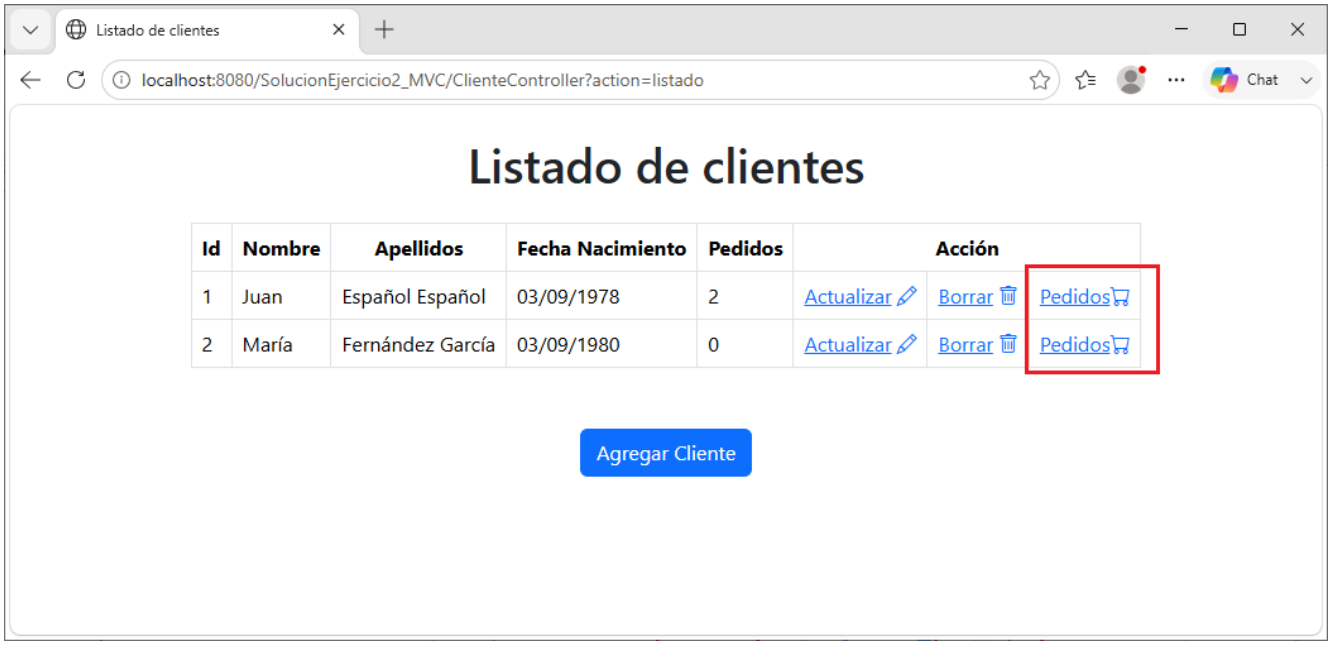
El propósito del ejercicio es permitir incrementar la funcionalidad del ejercicio resuelto del tema 4, para que la clase Cliente tenga una lista de pedidos y se pueda ir añadiendo pedidos al cliente.

Tomando como punto de partida el ejercicio resuelto del tema 4, se añadirán nuevas vistas y la lógica necesaria para registrar los pedidos que realiza un cliente.

Finalmente se añadirán las validaciones que eviten introducir datos incorrectos.

La funcionalidad añadida será la siguiente:

- En el listado de clientes, a las acciones de Añadir y Borrar se añadirá una tercera que será Pedidos, y que dirigirá al listado de pedidos del cliente:



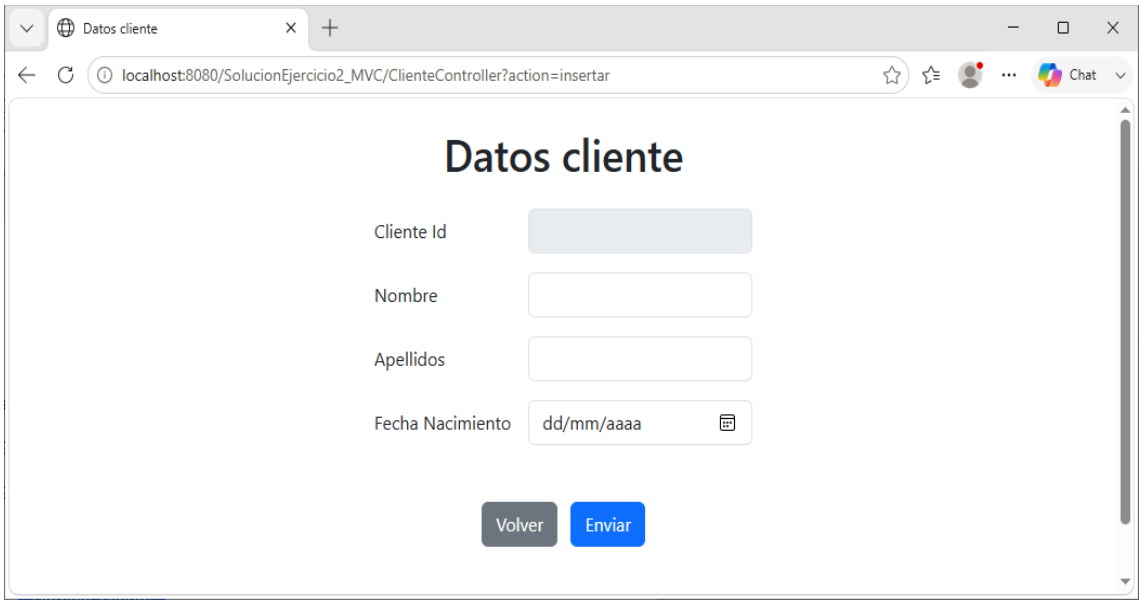
- La pantalla del listado de pedidos:
  - mostrará la lista de pedidos del cliente en una tabla, junto con el nombre del cliente.
  - tendrá un enlace / botón en la parte inferior que dirigirá a la pantalla de inserción de pedido.



- La pantalla de insertar pedido tendrá un formulario con los datos del pedido:



- La pantalla de insertar o actualizar un cliente no llevará el campo de número pedidos, ya que ahora será un campo calculado:



Nota: en el ejercicio resuelto se han aplicado estilos css usando la librería bootstrap, no obstante el propósito del ejercicio no es aplicar correctamente los estilos, el alumno debe centrarse en la parte funcional. **La aplicación de los estilos bootstrap es opcional** y no se tendrá en cuenta para la valoración del ejercicio.

RECORDATORIO: flujo de datos

Antes de detallar los pasos vamos a recordar el flujo de una petición http, que sería el siguiente:

1) El controlador, en el método *doGet* o *doPost*, recoge la información con `request.getParameter`. Esta información se la tenemos que haber pasado por la url si es un get, o en los campos del formulario si es un post.

2) El controlador procesa la información.

3) Dos opciones para el resultado:

   Opción 1: redirigir a un jsp. La opción a utilizar con peticiones GET.

   - El controlador guarda en el request, con `request.setAttributte`, la información que necesitará la vista (el jsp).
   - El controlador redirige con un *forward* a la vista (jsp).
   - El jsp utiliza la información que el controlador ha guardado con `request.setAttributte`, utilizando ${...}.

Opción 2: enviar un redirect al navegador con la url que queremos cargar (`response.sendRedirect`). Esto se puede hacer para evitar que al recargar la pagina se vuelva a enviar un POST y se creen clientes o pedidos duplicados. Esta opción es la que se debe usar en las peticiones POST.

Es importante tener claros estos conceptos antes de abordar el ejercicio.

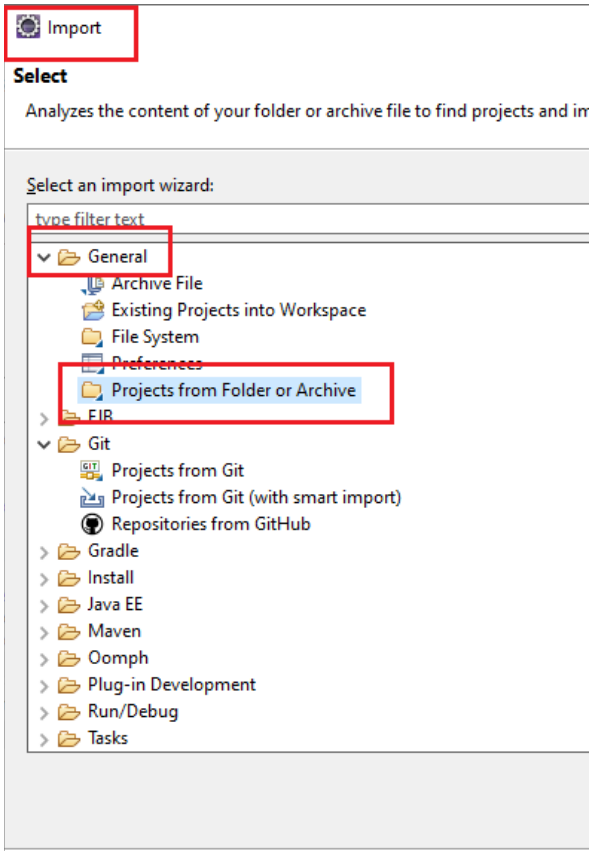
A continuación se explican paso a paso los cambios que hay que realizar.

PASO 0. Preparación inicial.

El ejercicio parte del ejemplo resuelto del temario (ProyectoMVC). Si el alumno ha realizado el ejercicio paso a paso, podría tomar su versión de ProyectoMVC como punto de partida para el ejercicio.

Otra opción es descargar de la plataforma el zip del proyecto Eclipse ProyectoMVC y utilizarlo como punto de partida. Se recomienda esta opción ya que en el zip se han añadido estilos css para trabajar con una interfaz más amigable.

Para importar el proyecto, debemos descomprimir el zip en una carpeta (no la del workspace de Eclipse), y después en Eclipse se hace desde el menú File → Import → Projects from folder or archive:



Pero si descomprimimos en la carpeta del workpace de Eclipse entonces hay que seleccionar la opción “Existing projects into Workspace”.

Una vez tengamos el proyecto importado debemos ejecutarlo para comprobar que no hay ningún error. De esta forma si posteriormente tenemos errores podremos descartar que fueran errores de configuración del proyecto original. Cuando podamos acceder al navegador y crear y editar clientes entonces pasaremos a modificar el proyecto, siguiendo los siguientes pasos.

PASO 1. Ampliación del modelo de dominio.

Se ampliará el modelo de dominio para que un cliente tenga un conjunto de pedidos. Cada pedido únicamente tendrá el nombre del artículo y el precio. El campo número de pedidos pasará a ser un dato calculado que devuelve la cantidad de pedidos que ha realizado el cliente.

Para ello:

- Se debe añadir una clase `Pedido` dentro del paquete *model*, con dos atributos:
  - `precio (BigDecimal)`
  - `articulo (String).`

  Se implementarán el constructor y los getter y setter correspondientes.

- Se debe modificar la clase Cliente:
  - añadir un atributo `pedidos` de tipo `ArrayList<Pedido>`, con su getter y setter.
  - eliminar el atributo `numPedidos` y el método `setNumPedidos`
  - modificar el método `getNumPedidos` para que devuelva el tamaño del atributo `pedidos`.

PASO 2. Adaptación del DAO.

El ejercicio resuelto empleaba la clase `ClienteDao` para guardar en memoria la lista de clientes. El funcionamiento era muy sencillo, ya que guardaba la lista en un `HashMap`, donde por medio del id de cliente era capaz de localizar al Cliente.

La clase `ClienteDAO` en principio no haría falta modificarla, ya que no insertaremos los pedidos directamente sino a través de los clientes.

El problema que surge ahora es que van a convivir dos controladores, uno para realizar las operaciones sobre el cliente y otro para realizar las operaciones sobre los pedidos de un cliente. Ambos controladores deberán trabajar con la misma instancia de la clase `ClienteDao` para garantizar que no hay dos listas de clientes.

Por lo tanto, hay que adaptar la clase `ClienteDao`, para que todos los objetos compartan la misma instancia de la clase. Para ello se deberá implementar el patrón *singleton*. Los patrones de diseño son soluciones habituales a problemas comunes en el diseño de software. El patrón *singleton* garantiza que haya una única instancia de un objeto para todas las clases que operan con él.

Existen diversas formas de implementarlo, una de ellas es la siguiente:

- Dentro de la clase `ClienteDao`, se crea un atributo estático y privado que contiene un objeto `ClienteDao`, y se inicializa.

- Se modifica el constructor de la clase `ClienteDao`, definiéndolo privado. De modo que sólo puede ser invocado por la propia clase.

- Se crea un método estático y público que permite obtener una instancia del objeto `ClienteDao`.

```
…
static private ClienteDao dao = new ClienteDao();

private ClienteDao() {
     bdClientes = new HashMap<Integer, Cliente>();
}

static public ClienteDao getInstance() {
    return ClienteDao.dao;
}
…
```

- Una vez se ha hecho esto, se deberá modificar el controlador de cliente para que obtenga el objeto `ClienteDao` a través del método `getInstance()`

```
public ClienteController() {
       super();
       dao = ClienteDao.getInstance();
}
```

PASO 3. Creación y actualización de clientes.

Como hemos mencionado, el número de pedidos ya no es un atributo del cliente, por tanto al insertar y actualizar el cliente no se debe incluir. Las modificaciones que hay que realizar son las siguientes:

- Modificar *cliente.jsp* eliminando el campo `numPedidos`.

- Modificar `ClienteController`, en el método *doPost*, para que no recupere el parámetro `numPedidos` ni lo establezca en el cliente. Observaremos que si no realizamos esta modificación se producirá un error de compilación, ya que hemos suprimido el método `setNumPedidos`.

  Por otro lado, hay que tener cuidado para que al actualizar un cliente no pierda los pedidos. La implementación del ejemplo creaba un nuevo cliente y reemplazaba el anterior por el nuevo, pero si ahora hacemos esto perderemos los pedidos. Para evitarlo, en caso de que estemos actualizando hay que recuperar el cliente del dao y modificarlo en lugar de sustituirlo por uno nuevo.

PASO 4. Modificación de la vista inicial (listado de clientes).

En la vista inicial (listado de clientes), debemos añadir en la tabla un enlace "Pedidos" para cada usuario. Para ello debemos editar *listadoClientes.jsp*, y dentro del bucle *forEach*, añadiremos un enlace similar a los otros dos, con la particularidad de que en lugar de dirigir a `ClienteController` dirigirá a `PedidoController`. El action lo denominaremos "listado", de forma que el enlace sería así:

PedidoController?action=listado&clienteId=1

Donde "1" es el id de cliente (varía para cada cliente).

Respecto a la columna que muestra el número de pedidos, obsérvese que no es necesario hacer nada: en *listadoClientes.jsp* se utiliza `${cliente.numPedidos}`, que lo que hace es invocar al método `getNumPedidos`. Nosotros hemos cambiado la implementación de este método pero sigue existiendo, así que la pantalla debe mostrar el número de pedidos correcto.

PASO 5. Implementación del listado de pedidos

En este paso deberemos crear la clase `PedidoController` y la vista *listadoPedidos.jsp*.

Se debe crear el controlador `PedidoController`, similar a `ClienteController`.

- El constructor inicializará el dao de la misma manera que `ClienteController`. Como ya se había indicado anteriormente ambos controladores deben trabajar con la misma instancia del objeto, en caso contrario cada controlador trabajaría con una lista de clientes independiente (de ahí la necesidad de implementar el patrón *singleton*). Por lo tanto, debe obtener la instancia de dicho objeto del mismo modo que se obtenía en `ClienteController`.

```
public PedidoController() {
        super();
        dao = ClienteDao.getInstance();
}
```

- Inicialmente sólo implementaremos el método *doGet*, que es el que nos permitirá listar los pedidos. Recordemos que la url que se estará invocando será de la forma *PedidoController?action=listado&clienteId=1*. Este método debe recuperar el parámetro "action", y en caso de que sea igual a "listado" realizar lo siguiente:
  - recuperar el parámetro *clienteId*.
  - obtener el cliente cuyo id sea el pasado por parámetro.
  - añadir al request ese cliente: `request.setAttribute("cliente", cliente);` De esta manera el *jsp* al que redirijamos tendrá acceso a un objeto con el nombre "cliente" que será el cliente deseado. El cliente ya incorpora los pedidos.

- Redirigir a la vista *listadoPedidos.jsp* (forward).

- Incluir el nuevo controlador en el fichero web.xml

  Para que Tomcat sea capaz de redirigir las peticiones al nuevo controlador (nuevo servlet) hay que mapear las peticiones. Posiblemente Eclipse haya añadido el nuevo controlador en el fichero web.xml (gracias a su asistente), sin embargo, conviene verificar que figura en el fichero. De modo que cuando se invoque a una url con el prefijo /PedidoController lo derive al nuevo servlet.

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee     http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
version="4.0">

  <servlet>

    <description></description>

    <display-name>ClienteController</display-name>

    <servlet-name>ClienteController</servlet-name>

    <servlet-class>controller.ClienteController</servlet-class>

  </servlet>

  <servlet>

    <description></description>

    <display-name>PedidoController</display-name>

    <servlet-name>PedidoController</servlet-name>

    <servlet-class>controller.PedidoController</servlet-class>

  </servlet>

  <servlet-mapping>

    <servlet-name>ClienteController</servlet-name>

    <url-pattern>/ClienteController</url-pattern>

  </servlet-mapping>

  <servlet-mapping>

    <servlet-name>PedidoController</servlet-name>

    <url-pattern>/PedidoController</url-pattern>

  </servlet-mapping>

  <display-name>ProyectoMVC</display-name>

  <welcome-file-list>

    <welcome-file>index.html</welcome-file>

    <welcome-file>index.htm</welcome-file>

    <welcome-file>index.jsp</welcome-file>

    <welcome-file>default.html</welcome-file>
```

```
        <welcome-file>default.htm</welcome-file>
        <welcome-file>default.jsp</welcome-file>
      </welcome-file-list>
    </web-app>
```

A continuación se deberá crear la vista *listadoPedidos.jsp*.

- En la parte superior mostraremos los datos del cliente. Recordemos que el controlador ha almacenado el cliente con el nombre "cliente", por tanto tendremos acceso a sus atributos mediante `${cliente.nombre}, ${cliente.apellidos}`, etc.

- En la parte central, mostraremos la tabla de pedidos. Será una tabla similar a la del listado de clientes: debemos iterar utilizando el tag `<c:forEach`, y el atributo *items* debemos establecerlo a `${cliente.pedidos}`. En el atributo *var* pondremos el nombre de la variable que se crea en cada iteración, si por ejemplo lo llamamos "pedido" entonces dentro del bucle podremos acceder a los atributos del pedido invocando `${pedido.articulo}` y `${pedido.precio}`.

  Nota: puesto que inicialmente los clientes no tienen pedidos, al principio veremos la tabla vacía, y no podremos comprobar el correcto funcionamiento hasta que implementemos la inserción de pedidos en el paso siguiente.

- En la parte inferior se debe incluir un enlace a insertar pedido. El enlace será de la siguiente forma:

  PedidoController?action=insertar&clienteId=1

  Donde *clienteId* es el id del cliente. Obsérvese que, a diferencia de los clientes, cuando se inserta un pedido es necesario pasar como parámetro el id del cliente, para saber a qué cliente estamos añadiendo el pedido.

PASO 6. Creación de pedidos

Para implementar la creación de pedidos haremos un formulario similar al de clientes:

- En primer lugar en el método *doGet* de `PedidoController` añadiremos un nuevo action. Si el action es "insertar", obtendremos el parámetro `clienteId`, y lo almacenaremos en el request para que esté disponible para la vista jsp (`request.setAttribute`). En este caso no es necesario recuperar el cliente del dao, basta con almacenar el id del cliente que es lo único que necesita la vista jsp. Después redirigiremos a la vista *pedido.jsp*.

- Crearemos la vista *pedido.jsp*, que contendrá un formulario con los parámetros del pedido (artículo y precio). A diferencia de los clientes, no vamos a utilizar este formulario para modificar pedidos, únicamente para insertar nuevos pedidos. El action del

formulario será *PedidoController*. Y en el formulario deberemos añadir un input hidden con el id del cliente, para que el controlador sepa el cliente al que hay que añadir el pedido.

- Cuando pulsemos el botón de submit, entraremos en el método *doPost* de `PedidoController`, método que debemos implementar. Este método además del artículo y el precio debe recuperar el id del cliente, que viene del input hidden. Recuperaremos del dao el cliente con el id indicado, y le añadiremos el pedido a su lista de pedidos. Por último redirigiremos al listado de pedidos del cliente, se podría redirigir directamente a la vista jsp, pero esto tiene el inconveniente de que si el usuario recarga la página, vuelve a enviar otro POST y puede duplicar el pedido. Para evitarlo, lo que se suele hacer es un response.redirect, que envía al navegador un redirect y el navegador carga mediante GET la url indicada, de forma que se puede recargar sin peligro.

Si todo funciona correctamente veremos que el número de pedidos se ha actualizado en la página de inicio, y el pedido se ha añadido a la lista de pedidos.

PASO 7. Pruebas

Se debe probar al menos insertar dos clientes, cada uno con varios pedidos, para ver que no se mezclan los pedidos de un cliente con los de otro.

También se debe probar que al actualizar un cliente no pierda los pedidos.

PASO 8. Opcional. Se incluirán las validaciones en los campos.

Este requisito es opcional: no se valorará para la nota, pero los alumnos que lo deseen pueden realizarlo y los tutores lo corregiremos.

Uno de los requisitos más comunes en las aplicaciones web es validar la información procedente de los formularios. Así se evita que el sistema genere errores de ejecución.

En el ejemplo del que se parte se han incluido unas validaciones a nivel de cliente utilizando las funcionalidades que proporciona html5 (atributos required, input date, etc). Estas validaciones las realiza el navegador. Sin embargo toda aplicación robusta debe implementar también las validaciones a nivel de servidor, ya que el usuario podría desactivar las validaciones del navegador, o incluso podríamos recibir peticiones generadas no por un navegador convencional sino por un software malicioso.

Existen diferentes formas de validar los formularios:

- El mecanismo más rudimentario es añadir en los controladores que reciben los datos del interfaz de usuario una serie de *if*, verificando que los objetos recibidos cumplen las validaciones indicadas.
- Otro mecanismo más sofisticado es emplear alguna implementación de la especificación JSR 380, Bean Validation, que permite indicar en las clases de dominio (en nuestro caso Cliente y Pedido) las validaciones que deben cumplir estos objetos a

través de anotaciones, de modo que en el controlador únicamente habría que verificar si el objeto cumple las restricciones establecidas.

Dado que el uso de Bean Validation no es trivial, el alumno podrá realizar las validaciones como desee.

Las validaciones que se deberán implementar son:

- Obligar a cumplimentar todos los campos.
- Comprobar el formato de la fecha, para evitar introducir fechas con formato incorrecto. Para ello nos podemos apoyar en las expresiones regulares.
- Comprobar el formato numérico del precio del pedido.

Finalmente se mostrarán las validaciones por pantalla.

Nota.

Si el alumno opta por emplear una implementación de la especificación Bean Validation, se propone el uso de Hibernate Validator.

Para ello, se suben a la plataforma las librerías requeridas:

- hibernate-validator-7.0.5.Final.jar (Implementación de la especificación)
- jakarta.validation-api-3.0.2.jar (Especificación bean validator)
- classmate-0.8.0.jar (jar requerido por hibernate validator)
- jboss-logging-3.5.0.Final.jar (jar requerido por hibernate validator)

Bastaría con añadirlas al directorio webapp/WEB-INF/lib.

Posteriormente habría que añadir las validaciones sobre las clases del modelo de dominio por medio de anotaciones (constraints).

Por ejemplo, para el atributo nombre de la clase Cliente:

```
@NotNull(message = "El nombre es obligatorio")  // Verifica que el campo no es nulo
@NotBlank(message = "El nombre es obligatorio") // Verifica que el campo no está en blanco
private String nombre;
```

Y posteriormente habrá que validar los objetos en el controlador, a través de un objeto que implemente el interfaz Validator, que permitirá verificar si el objeto cliente cumple las validaciones.

```
Validator validator = Validation.buildDefaultValidatorFactory().getValidator();
Set<ConstraintViolation<Cliente>> violations = validator.validate(cliente);
```

Si los objetos cumplen todas las validaciones, el conjunto resultante es vacío, en cambio si el objeto no cumple alguna validación, las restricciones que no se cumplen figurarán en dicho conjunto.

Y finalmente, habría que obtener los mensajes de error, en caso de haberlos, y mostrarlos por pantalla.

Para más información se puede consultar el *getting started* que ofrece el propio hibernate al respecto:

- Aplicando restricciones: Applying constraints
- Validando las restricciones: Validating constraints

O el API de Bean Validation: Overview (Jakarta Bean Validation API 3.0.0